

AML Compliance Multi-Agent Desk

Solution Approach & Architecture Overview AI Agent Developer Assignment Submission

Repository: <https://github.com/Mohitmeenna/aml-compliance-agents>
Eval status: 15/15 pass (pass_rate = 1.0) after bootstrap

1. Problem & Goal

Build a working agentic system for a bank AML / transaction-monitoring desk. The system must ingest heterogeneous compliance data (regulatory PDFs + tabular ledgers/sanctions), convert it into artifacts agents can reason over, enforce role-based access at the data layer, answer natural-language questions, and improve over time from analyst alert dispositions. The guiding constraint from the brief: a smaller system that runs end-to-end beats an elaborate design that does not.

2. High-Level Architecture

The solution is a Python multi-agent pipeline with a single data-access choke point (DataGate). Every question is scoped to a user profile; agents never see raw tables directly.

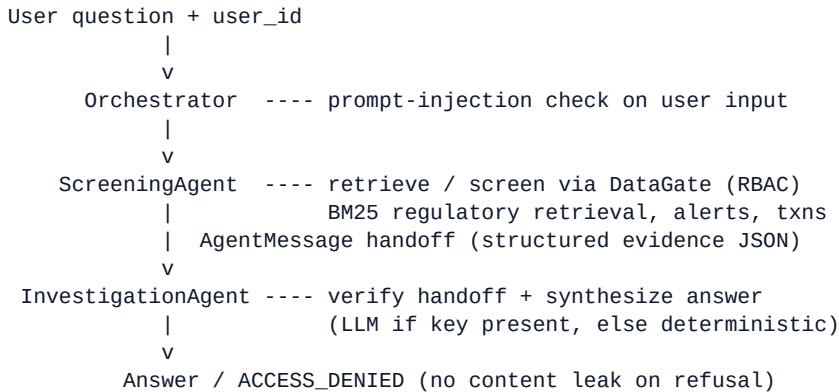


Figure: request path from question to answer, with RBAC applied before synthesis.

2.1 Agent boundaries

- **ScreeningAgent** — intent detection, retrieval, sanctions/structuring/correspondent screening assembly. Owns evidence gathering; fails closed on AccessDenied.
- **InvestigationAgent** — consumes the handoff, verifies consistency (drops malformed alerts, notes missing buckets), drafts the final answer from evidence only.
- **Orchestrator** — wires the handoff, blocks instruction-injection style user queries, short-circuits refusals so protected content never reaches synthesis.

When screening fails softly, investigation continues in degraded mode and surfaces the screening error instead of inventing evidence. That makes the handoff observable and matches how a real desk separates screening from investigation write-up.

3. Data & Ingestion

Public / synthetic sources only — no real customer PII and no employer-confidential data.

Source			Format	Notes
Customer	master	+	CSV / XLSX	Synthetic seeded AML patterns (structuring, sanctions hit, correspondent flags)
Sanctions / watchlist			CSV / XLSX	Synthetic subset styled after OFAC/UN fields (self-contained, license-clean)

Regulatory guidance	PDF	Synthetic excerpts paraphrasing public FATF / FinCEN / RBI themes; real PDF files under data/raw/regulatory/
SARs + audit trail	JSON	Synthetic filings and disposition events for RBAC demos

PDFs are chunked on Section/Chapter/Advisory headings to preserve hierarchy rather than blind fixed-size splits. Tabular files are normalized through pandas with NaN cleanup.

4. Understanding Layer

Raw files are converted into derived artifacts under **data/understanding/** so agents do not re-parse PDFs and re-score the ledger on every question.

Precomputed

- Regulatory chunks + BM25 token index
- Obligation / threshold extraction (e.g. CTR ~ USD 10,000; structuring window)
- Counterparty activity profiles
- Baseline screening alerts with feedback-adjusted score multipliers

Computed on the fly (per request)

- DataGate filtering and PII masking for the active role
- Query intent routing and evidence assembly
- Answer synthesis (LLM or deterministic)
- Disposition writes that rebuild multipliers

Why this split: parsing and full-ledger screening are relatively stable and expensive; access control must stay request-scoped so a cached CCO answer can never be reused for an auditor.

5. Role-Based Access Control (data layer)

Permissions live in **config/roles.yaml** — not only in prompts. **src/rbac/DataGate** is the only path agents use to read customers, transactions, alerts, SARs, sanctions, audit events, and regulatory chunks.

Profile	Role	Access sketch
u_cco	Chief Compliance Officer	Full access including PII and SARs
u_analyst	AML Analyst	Alerts/txns with masked PII; no SARs / ID docs
u_auditor	External Auditor	Audit trail + disposition rationale; no raw txn/customer PII
u_rm	Relationship Manager	Portfolio PF-APAC-01 only

- Refusals return a standard ACCESS_DENIED message with **no protected payload**.
- Masking rules (partial name, last-4 accounts, redacted IDs) apply when a role can read a record but not PII.
- Aggregates / clever phrasing cannot bypass the gate: the model only receives already-filtered JSON.

6. Feedback & Learning Loop

Analysts disposition alerts as **true_hit**, **false_positive**, or **escalate**. Each disposition updates a per-feedback_key score multiplier and appends a few-shot memory entry; understanding/alerts are then rebuilt so future screening scores change.

- Demo command: `python -m src.query.cli feedback-demo`
- Observed before/after on a fuzzy sanctions candidate: multiplier 1.0 → 0.65; alert score 0.95 → 0.617
- This is behavioural change, not only logging a thumbs-down.

7. Query Interface & LLM

The interface is a minimal CLI (`python -m src.query.cli ask ...`). The assignment explicitly allows CLI / simple UI; correctness matters more than polish.

- OpenAI is **not** required. Any OpenAI-compatible API works (Groq free tier tested with llama-3.3-70b-versatile).
- Without a key, InvestigationAgent uses a deterministic synthesizer — sufficient for RBAC, screening, and eval.
- With a key, the same RBAC-filtered evidence JSON is passed to the model for a clearer investigation-style answer.
- Bonus: remittance narratives and override-style user queries are sanitized / blocked (prompt-injection surface).

8. Evaluation Suite

Ship-with: `eval/eval_set.json` (15 cases) and `python -m src.eval.runner`. Role scopes are enforced in code before the model sees data. Current reported pass rate after bootstrap: **15/15 (1.0)**.

Coverage includes:

- Admin (CCO) + two non-admins (analyst, auditor) + portfolio-scoped RM
- At least one RBAC refusal path per restricted profile family
- Questions that answer differently by profile (e.g. unmasked PII)
- Questions needing both regulatory document retrieval and transaction/alert evidence
- Injection / override attempt that must not leak national IDs

Checks are structural (denial flags, required/forbidden substrings, leak tests, regulatory+txn evidence presence) so the suite stays honest under LLM wording variance.

9. What Breaks at 100×

Three concrete failure modes if this moved from the sample set to bank-scale volume:

- **In-memory alert rebuild** — full ledger rescreen on feedback becomes multi-minute and memory-bound under millions of txns/day. Change: streaming screening + durable alert store; feedback as feature multipliers without full recompute.
- **Process-local BM25** — thousands of pages across jurisdictions and hundreds of concurrent users need a shared search service. Change: OpenSearch/Vespa with jurisdiction collections and ACL filters at query time.
- **Per-request full-table DataGate filtering** — loading then masking entire tables in Python does not scale and risks touching rows a role should never load. Change: DB row-level security / masked views and scoped queries only.

10. How to Run (for reviewers)

```
pip install -r requirements.txt
copy .env.example .env          # optional: add Groq/OpenAI-compatible key
python -m scripts.bootstrap
python -m src.eval.runner
python -m src.query.cli ask --user u_cco -q "Which transactions this month breach the FATF guidance on co
python -m src.query.cli feedback-demo
```

11. Deliverables Map

- Working code + README with setup and 100× section — repository linked above
- Eval set + runner + pass-rate note — `eval/` and README
- Understanding artifacts — `data/understanding/` (regenerable via `scripts/build_understanding.py`)
- Walkthrough — live call (talk track: `WALKTHROUGH_SCRIPT.docx`) or recorded video
- This PDF — concise written approach for email attachment